Language Processor Practice

Definition of the C-Horizon language
Academic Year 2025–2026

Author

Rafael Moreno Portilla
Antonio Parrilla Sánchez

Double Degree in Computer Science and Mathematics
Faculty of Computer Science
Complutense University of Madrid

Contents

1 Identifiers and Definition Scopes in C-Horizon **1**
1.1 Variable Declarations 1
1.2 Nested Blocks 2
1.3 Modules and Import Clauses 2
1.4 Functions 3

2 Types and Operators in C-Horizon **5**
2.1 Integer 5
2.2 Boolean 5
2.3 Void 5
2.4 Array 5
2.5 Struct 6
2.6 Class 7
2.7 Pointers 8
2.8 Functional 8
2.9 Arithmetic Operators (for integers) 9
2.10 Logical Operators (for booleans) 10
2.11 Comparison and Assignment Operators 11

3 C-Horizon Instruction Set **13**
3.1 Assignment 13
3.2 ; 13
3.3 While 14
3.4 For 14
3.5 If, elsif and else 14
3.6 Switch 14
3.7 Input and Output 14

4 Error Handling in C-Horizon **17**
4.1 Lexical Errors 17
4.2 Syntax Errors 18
4.3 Name Binding Errors 19

4.3.1 Identifier Already Declared 21
4.3.2 Identifier Not Declared 21
4.3.3 New Type Already Defined 22
4.3.4 New Type Not Defined 22
4.3.5 Error in Number of Arguments 23
4.3.6 Missing main Error 23
4.3.7 Multiple main Error 24
4.4 Type Checking Errors 25

5 Code Generation **27**
5.1 Code Examples 27
5.1.1 Example 1 27
5.1.2 Example 2 27
5.1.3 Example 3 28
5.1.4 Example 4 29
5.1.5 Example 5 30
5.1.6 Example 6 32
5.1.7 Example 7 33
5.1.8 Example 8 34
5.1.9 Example 9 35
5.1.10 Example 10 36
5.1.11 Example 11 37
5.1.12 Example 12 38
5.1.13 Final Notes 38

Chapter 1

Identifiers and Definition Scopes in C-Horizon

1.1 Variable Declarations

To declare a variable, the process is very similar to C++. When declaring, we usually write the variable type and then its name. Let us now discuss pointer, array, and functional variable declarations. For pointers, for each type T we can create the pointer-to-T type denoted as $T*$. Likewise, for each type T and natural number n, we have the array type of elements of type T and size n, denoted by $T[n]$. Finally, for each type T and each ordered tuple of types, $Targ_1, \ldots, Targ_m$, we have a functional type of functions that return a value of type T, have m parameters, and whose i-th argument has type $Targ_i$. We denote this functional type by $T(Targ_1, \ldots, Targ_m)$.

A variable name cannot be any arbitrary character string; we only allow words whose first character is one of $["a" - "z"] \mid ["A" - "z"]$ and whose remaining characters belong to $["0" - "9"] \mid ["a" - "z"] \mid ["A" - "z"] \mid "_"$.
In addition, we allow a variable to be declared and instantiated with a value at the same time, as long as it is not in the global scope. Functions must always be instantiated when declared; this was not initially required, but it was done this way to simplify later phases. [1]

[1] The examples in the following chapters are illustrative only and may contain compilation errors. In the error management and code generation chapters we will show real code examples. Each of these examples comes from a text file in the tests folder.

```
int  X := 2;//Declaration and assignment to an integer variable
bool  a := True;//Declaration of a boolean variable
int*  p := null;//Declaration and assignment of an integer pointer variable
//Declaration and assignment of a boolean array
bool[3]  Y := [True, True, False] ;
int (int ) f(a) { //Declaration of a functional type
  return  a;
};
//Functional types may return the special type void
void (int ) f(a) {};
```

1.2 Nested Blocks

Each correctly placed pair { } defines a scope in our language and, conversely, each scope is delimited by this pair, except for the global scope. Each { } pair must appear after a context that allows it, such as an if + condition or a function declaration with its arguments. Each scope may have local variables, which are lost when leaving that scope, and its own symbol table. In addition, as expected, nested scopes are allowed, i.e., subscopes inside a larger scope (the superscope), defined through nesting of { } or, if the superscope is the whole program, by including { } as long as the subscope does not come from a function or a class/struct definition. In this case, there is no visibility outside these scopes except for method declarations in a class and its attributes. Therefore, the symbols in the subscope form a superset of the symbols in the superscope. In addition, shadowing of superscope variables is allowed by declaring new variables with the same name in the subscope.

1.3 Modules and Import Clauses

Our language was intended to allow external modules, which are simply code files. However, this was not implemented, not even during lexer creation, since it was given the lowest priority.

1.4 Functions

Functions are declared with a functional type, an identifier, an argument list, and the function body (a scope ending in ;). Functions may also be defined in any scope, although they will not be visible outside that scope.

Each program has a special function named main, with argument types undefined until we create it, and return type void, since it is only a special symbol.

By default, argument passing is by reference and becomes by value if explicitly indicated with the $operator. However, this operator is syntactic sugar; the type does not change, since it only tells us to copy each by-value argument into a new auxiliary variable and operate on that new variable as if it were the original one. Still, not all types allow pass-by-value, such as any functional type. We also support function overloading if the functional type differs (in the return type and/or parameter types). The exception is the "main" function, which may only be declared in the global scope and must be declared exactly once.

Chapter 2

Types and Operators in C-Horizon

In this section we specify the types and operators we will use. In case of precedence ties, we always associate to the left. We should also emphasize that we removed typedef for defining our types because it caused grammar conflicts.

2.1 Integer

It corresponds to C++ *signed int*. That is, it stores integer numbers represented in decimal form. It reserves 32 bits, so its value range is [-2147483648, 2147483647]. The reserved word for this type is `int`.

2.2 Boolean

It corresponds to C++ *bool*. It stores true (True) or false (False) values. Automatic type conversion from `int` is performed, where 0 represents false and any other integer represents true. The reserved word for this type is `bool`.

2.3 Void

This is the empty type. We use it when a function or method does not return any value. Its reserved word is `void` .

2.4 Array

It corresponds to C++ arrays. To declare one, we write the type followed by $[n]$, where n is a fixed natural number representing the size. Its elements are accessed in the same way, using the [] operator. They can be assigned directly with brackets (recursive definition allowed) or with another array variable. The following examples detail different uses:

```
void () main () {
  int[1]  a; //Declaration of a single-element array
  int[5]  b; //Declaration of a five-element array
  int[3]  c := [2, 3, 4]; //Declaration of an initialized array
  int[3][2]  d := [[2,1], [3,3], [4,4]]; //Two-dimensional array (array of array type)
  a[0] := a[0]+a[0]; //Access to uninitialized content (garbage)
  b[0] := 2; //Assign 2 to the first position of array b
  c[2] := 6; //Replace 4 with 6
  d[5] := 7; //Type error
  d[1] := [2 , 2] ; //Assign array [2,2] to the second position of d
};
```

[] Operator

Given an array type $t[n]$, the `[]` operator maps each array object and each in-range integer to the element at that position. We have also seen that it is used as a partial functor to define new types or instantiate arrays.
Binary operator.
Typing: `[]` :: $(t[n], 1, \ldots, n-1) \rightarrow t$.
Priority: 12.
Associativity: left.

2.5 Struct

We use the reserved word `struct` both to declare a record and to instantiate it. This is done similarly to C++. Records can contain only attributes and/or definitions of other structs (as expected). Attributes are accessed with the "." operator (without quotes). Inside a struct, only declared struct variables or globals can be accessed, as well as functions (declared outside, logically). This is an example:

```
struct  tEstudiante {
  int  edad;
  int  id;
};
void  main () {
struct  tEstudiante antonio := {22, 67532};
  antonio.edad := antonio.edad + 1; //Age increased by 1
};
```

2.6 Class

We use the reserved word `class` to declare a class. This is done similarly to C++. Classes have attributes and methods (functional attributes), all of them public. They do not have constructors; instead, attributes are initialized in the same way as in a struct: by directly accessing them or with a brace-list expression. Other methods and attributes are accessed with the "." operator (without quotes), followed by the method or attribute name. Depending on whether the expression is functional or not, an attribute or a class method is selected.

This is an example:

```
class Estudiante {
  int edad;
  int id;
  void incr_edad() { edad := edad + 1; };
};
void () main() {
  Estudiante antonio := {22, 67532};
  antonio.incr_edad(); //Age increased by 1
  return 0;
} ;
```

. Operator

Given a class or struct type t with attribute/method set A_t, the . operator maps each struct/class object and each attribute/method of that struct/class to the value of that field in that instance. If B_t is the union of the types of attributes/methods of t, then
Binary infix operator.
Typing: . :: $(t, A_t) \to B_t$.
Priority: 12.
Associativity: left.

2.7 Pointers

To declare a pointer to a variable, we simply use the * operator next to the type. To access the pointer value, we also use this operator. When a pointer is declared without instantiation, it takes value null (reserved word), as in C++.

```
int * p; //A pointer is declared
*p := 5; //Value 5 is assigned
int  num := *p; //Pointer value is accessed
```

* Operator (pointers)

Besides its use in multiplication, this symbol lets us relate a type to pointers of that type and, as a pointer operator, it also gives the value stored in each pointer.
Unary prefix operator.
Typing: *:: *t → t.
Priority 11.
Associativity: associative.

2.8 Functional

As stated before, functional types are determined by the return type and by the types of the ordered argument tuple. To obtain the value for given function arguments, we use the () operator. Inside a function scope, we may use the reserved word `return` to return a value. If the return type is not void, the function scope must satisfy a property we call returnable. A scope is returnable if it contains a return outside all its own subscopes, or if all its subscopes are returnable. We do not allow function prototypes; the function body must always be defined, although definitions do not need to appear in order. Shown below are examples:

```
int(int, int) suma(a,b) {
  return a + b;
} ;
int(int, int) division_entera(a,b) {
  if  (b =/= 0)  {
    return  a;
  }
  return  0 ;
};
void () main () {
  suma(0, 1); //We invoke function suma
};
```

() Operator

Given a functional type $t(ta_1, \ldots, ta_n)$, the `()` operator maps each function and arguments to a return value, that is, if f is a function of type $t(ta_1, \ldots, ta_n)$ and a_i has type ta_i for $i \in \{1, \ldots, n\}$, then `()` has arity $n + 1$ in this case and $()(f, a_1, \ldots, a_n) = f(a_1, \ldots, a_n)$. In addition, we have seen that it also allows us to define new functional types, although not as an operator itself.
Binary operator if arguments are considered as one.
Typing: `()` :: $(t(ta_1, \ldots, ta_n), ta_1, \ldots, ta_1) \rightarrow t$.
Priority: 12.
Associativity: left.

Pass by Value

Indicates that the function argument is passed by value. It is syntactic sugar; in practice it only means we must copy the argument inside the function scope and use that copy as the new argument. Its pseudo-operator is `$` because as syntactic sugar it is not a real operator.
Typing: `$`:: t → t.
Priority 1.
Associativity: non-associative.

```
//A function with two integer parameters is declared: one by value and one
  by reference (reference is the default)
int ($ int , int) sumaNum(a, b) {
  return a + b;
} ;
```

2.9 Arithmetic Operators (for integers)

Plus

Sum of two integers. The `+`operator is used.
Binary infix operator.
Typing: `+`:: (`int`, `int`) → `int`.
Priority: 7.
Associativity: left.

Positive integer. The `+`operator is used.
Unary prefix operator.
Typing: `+`:: `int`→ `int`.
Priority: 11.
Associativity: associative.

Minus

Subtraction of two integers. The `-`operator is used.
Binary infix operator.
Typing: `-`:: (`int`, `int`) → `int`.
Priority: 7.
Associativity: left.

Additive inverse of an integer. The `-`operator is used.
Unary prefix operator.
Typing: `-`:: `int`→ `int`.
Priority: 11.
Associativity: associative.

Multiplication

Multiplication of two integers. The `*`operator is used.
Binary infix operator.
Typing: `*`:: (`int`, `int`) → `int`.
Priority: 8.
Associativity: left.

Division

Integer division of two integers. The result is truncated if fractional. The operator used is `/`.
Binary infix operator.
Typing: `/`:: (`int`, `int`) → `int`.
Priority: 8.
Associativity: left.

2.10 Logical Operators (for booleans)

Conjunction

Logical conjunction of two booleans. Its infix operator is `&&`.
Binary infix operator.
Typing: `&&`:: (`bool`, `bool`) → `bool`.
Priority: 4.
Associativity: left.

Disjunction

Logical disjunction of two booleans. Its infix operator is `||`.
Binary infix operator.
Typing: `||`:: (`bool`, `bool`) → `bool`.

Priority: 3.
Associativity: left.

Negation

Logical negation of a boolean. Its prefix operator is `not`.
Unary prefix operator.
Typing: `not`:: `bool`→ `bool`.
Priority: 5.
Associativity: associative.

2.11 Comparison and Assignment Operators

Equal To

Compares two values of the same type and returns True if they are equal; otherwise it returns False.
Binary infix operator.
Typing: `==`:: (t, t) [1] → `bool`.
Priority 3.
Associativity: non-associative.

Not Equal To

Equivalent to `not`(a `==`b).
Binary infix operator.
Typing: `=/=` :: (t, t) → `bool`.
Priority 6.
Associativity: non-associative.

Less Than

Compares two values of the same type and returns True if the first is less than the second; otherwise it returns False [2]. Its infix operator is `<`.
Binary infix operator.
Typing: `<`:: (t, t) → `bool`.
Priority: 6.
Associativity: non-associative.
newpage

[1] We use "t" to denote any type
[2] We consider False to be less than True

Greater Than

Compares two values and returns True if the first is greater than the second; otherwise it returns False. Its infix operator is >.
Binary infix operator.
Typing: >:: (t, t) → bool.
Priority: 6.
Associativity: non-associative.

Less Than or Equal To

Equivalent to not(a <b).
Binary infix operator.
Typing: <= :: (t, t) → bool.
Priority: 6.
Associativity: non-associative.

Greater Than or Equal To

Equivalent to not(a >b).
Binary infix operator.
Typing: >= :: (t, t) → bool.
Priority: 6.
Associativity: non-associative.

Assignment

Assigns a value to a variable. The assigned value type must match the variable type. Its infix operator is ":=" (without quotes).
Binary infix operator.
Priority: 1.
Associativity: non-associative.

```
int num := 5; //Valid assignment
int num2 := True; //Invalid assignment due to type mismatch
```

Chapter 3

C-Horizon Instruction Set

3.1 Assignment

Our first kind of instruction is assignment. As we have seen throughout the document, there is varied syntax depending on the variable type. Examples from across the document are shown. Recall that assignments are not allowed in the global scope of the language.

```
int  X := 2;
int*  p := null;
p := new ;
*p := 2
bool[3]  Y := [True, True, False] ;
int[2][2]  Z := [[0,1],[2,3]] ;
int (int , int ) potencia(a,b) {
  if (b < 0) {
    return 0;
  } elsif (b == 0) {
    return 1;
  } else {
    return potencia(a,b-1);
  }
};
```

It only remains to comment on the `new` function, which is an overloaded function that allows dynamic memory allocation for a pointer.

3.2 ;

As in C++, ; is used to join two instructions into one.

3.3 While

The next kind of instruction is formed by a while loop. The reserved word `while` is used and the syntax is the usual one,

while (boolean expression) { instruction } and, in addition, we enforce the use of { and } to make its scope explicit.

3.4 For

The next kind of instruction is formed by a for loop. The reserved word `for` is used and the syntax follows standard C++. As before, we enforce the use of { and } to describe scope. In addition, the step condition can be written with increment and decrement operators.

3.5 If, elsif and else

The if, elsif, and else instructions use the usual syntax with reserved words `if`, `elsif`, and `else`, respectively. As in all previous cases, using { } is mandatory to declare the scope of each block.

3.6 Switch

Now we examine the next instruction type, those formed by a switch. We use the reserved words `switch`, `case`, and `default`. Using break is not necessary because each switch case is independent, but placing a final default clause is mandatory. The switch instruction evaluates a boolean expression only once and searches through cases for the one that matches that evaluation.

3.7 Input and Output

Finally, the last instruction type. For standard output we reserve the word `prints`, which prints the contents of a primitive variable to standard output. For standard input, we have the reserved word `scans`, which similarly reads the value of a given primitive variable from standard input.

Finally, we show an example that uses all instructions to illustrate their use: [1]

[1] Recall that this is illustrative; in the code generation section we will show fragments that can be compiled

```
int X := 100;
int Y := 0;
int Z := 1;
int T ;
int(int, int) f(a,b) {
  if (2*a < b) {
    return 2*a + 1;
  } elsif (a > b) {
    return b;
  } else
    return a + 1;
  }
};
for (Y := 0; Y < X; Y := f(Y,X)) {
  for (T := 0; T < Y; T := T + 1) {
    if (2*Y < X) {
      Z := Z + 1;
    }
  }
}
int Y := X - Z;
while (Y < X) {
  scans(T);
  switch(T - 2*Y)
    case 0 {
      T := X;
    } case 1 {
      T := 2*X;
    } case 2 {
      T := 0;
    } default {
      T := Y;
    }
  }
  Y := Y + 1;
};
prints(Y);
```

Chapter 4

Error Handling in C-Horizon

For each error, its type and position in the file (row and column) will be reported. The class GestionErroresExp is responsible for acting according to each error. In addition, it keeps an error counter that is printed when any error has occurred. Otherwise, it allows code generation. Let us explain the different kinds of errors.

4.1 Lexical Errors

The unrecognized token is simply printed.

Input [input6_errlexico.txt][1]

```
int (int ) main(a) {
  int  hola ::= 2; //Lexical error
  while  (True) {
    hola++;_//Lexical error
  }
  return a;
};
```

Output

- `[Row 2, Column 11] Unexpected character:  ":"`
- `[Row 4, Column 10] Unexpected character:  "_"`

[1]Whenever we provide an input to show output, we indicate the text file name in this way to execute the code. These files are located in the tests folder.

4.2 Syntax Errors

If the error appears in a simple instruction, compilation is resumed right where the instruction ends, i.e., after ";". If the error appears in a scope declaration (a function, a condition, a loop), compilation is resumed exactly when that scope ends. That is, the character "{" is searched for, and all code is skipped until finding the closing "}" (for this, we also keep a counter of unmatched "{"). Some examples of syntax errors:

Input [input7_errsintaxis.txt]

```
1 int (int ) main(a) {
2   int  hola := 2;
3   while  (int ) { //Syntax error in the condition
4     hola++;
5   }
6   return .;//Syntax error in the condition
7 };
```

Output

- [Row 3, Column 9] Unexpected element ‘‘int’’
- [Row 6, Column 9] Unexpected element ‘‘.’’

4.3 Name Binding Errors

All AST tree nodes implement two functions: bindFunc and bind. The first one, bindFunc, only inserts an entry into the current symbol table for a function declaration. We do this to allow a function declared in a block to be used anywhere inside that block. Therefore, each time we open a new block, we add a new symbol table, then always call bindFunc first and bind later on the program in that block. At the end of binding the block, we pop the symbol table. In addition, when popping, we store the block table because it is useful later (for struct/class definitions and type checking). The tables are implemented as maps with key String and value Declaracion. Instead of using one stack of tables, we used a stack of stacks of tables. There are several reasons. EXPLAIN THIS HERE.

When, using bind, we reach a variable declaration, we insert that declaration into the current symbol table. While inserting a variable declaration, we check that another variable declaration with the same identifier has not already been inserted. It is allowed to declare a variable and a function with the same identifier. Since overloading also exists, in binding we allow two function declarations to have the same identifier. During type checking we will verify whether both have the same functional type (and raise an ambiguity error if so).

When using an identifier, we search backward through the table stack and add to the identifier's binding set all matching declarations. When a variable declaration with a matching name is found in a block, we do not continue searching for variable declarations in previous blocks (shadowing). However, we do continue searching for function declarations that can be bound to that identifier.

Finally, in parallel we also maintain another stack of tables that collects newly declared types (classes or structs) in each block. When a new type is defined, we insert a new entry in the current type table. This table works exactly like the previous one, but its pairs are String (new type name) and NodoTipo (the concrete type).

Let us see an example of code that uses what we have just explained and produces no errors.

Input [input13_errvinc.txt]

```
int (int ) f (a) {};
bool  f;
int () main(){
  int  f; //Declaration of variable "f" that does not raise an error due to shadowing
  int (bool ) f (a) {};
  //The following does not raise a declaration or type error because f is correctly bound to the declaration in the first block
  int  x := f(2);
  //The following also does not raise a type error because f has been correctly bound to the declaration in this block
  x := f;
};
```

Next, we will see concrete binding error types.

4.3.1 Identifier Already Declared

This error is raised when an identifier is declared and it has already been declared.

Input [input8_errvinc.txt]

```
1 int hola;
2 bool hola; //Error: identifier already declared
3 int (int ) hola (a) ;
4 int (int ) hola (a) ; //This becomes a type error, not a binding error
5 void () main () {};
```

Output[1]

- `[Row 2, Column 6] Identifier already declared: hola`

4.3.2 Identifier Not Declared

This error is raised when an identifier is used but has not been previously declared.

Input [input9_errvinc.txt]

```
1 int () main() {
2   int x;
3   x := a + 1; //Binding error because "a" has not been declared
4   x := f(a); //Only undeclared-id error on a
5   {
6     int a;
7     x := a + 1; //This instruction is correct
8   }
9   x := a + 1; //Now it fails again because the block is closed
10 };
11 int (int ) f (num) ;
```

Output

- `[Row 3, Column 6] Identifier not declared: a`
- `[Row 4, Column 8] Identifier not declared: a`
- `[Row 9, Column 6] Identifier not declared: a`

[1] Only binding errors are included here

4.3.3 New Type Already Defined

This error is raised when a new type is defined with a name that had already been used to define another new type. Even if they are different kinds of new types (a class and a struct), if both have the same name we raise an error because it may cause ambiguity in the future.

Input [input10_errvinc.txt]

```
1 class A {
2   int  num;
3 };
4 struct A {}; //Error because a new type named "A" was already defined
5 class A a; //This declaration is correct, since only the first definition of
   "A" is accepted
6 int () main() {
7   a.num := 2;
8 };
```

Output

- `[Row 4, Column 8] New type already defined with that name:  A`

4.3.4 New Type Not Defined

This error is raised when a variable is declared with a new type that has not been declared previously.

Input [input11_errvinc.txt]

```
1 struct A a; //Error because that new type has not been defined
2 struct A {}; //Now it is declared; declarations will not fail
3 struct A a1;
4 int  () main () {
5   struct A a2;
6   struct B {};
7 };
8 struct B b; //Error because no visible declaration of struct B exists
```

Output

- `[Row 1, Column 1] New type not defined:  A`
- `[Row 8, Column 1] New type not defined:  B`

4.3.5 Error in Number of Arguments

This error is raised when a function is declared and the number of arguments does not match the number of function argument types. We detect this error during binding because when the function header is bound, declarations are inserted in the symbol table for each argument so that identifiers in the function scope can be bound to those declarations. To prevent these declarations from colliding with other declarations in that block, when binding function arguments we open an intermediate block between the block where the function is defined and the function body block.

Input [input12_errvinc.txt]

```
int (int ) main (a,b) {};
```

Output

- `[Row 1, Column 10] Error in the number of function arguments:  f`

4.3.6 Missing main Error

If a main function is not declared in the global scope, an error is raised. It must be in global scope. The syntax prevents it from being declared in any other scope.

Input [input16_errNoMain.txt]

```
//There is no main function
int (int , int ) suma (a,b) {
  return a + b;
};
int (int ) factorial (n) {
  if (n == 0) {
    return 1;
  }
  return n * factorial(n-1);
};
```

Output

- `[ERROR] Function "main" was not declared`

4.3.7 Multiple main Error

Multiple main functions were detected, which prevents code generation.

```
//Error: multiple main functions
void () main() {};
int (int ) main(arg){};
```

Output

- `[ERROR] Multiple "main" functions were declared`

4.4 Type Checking Errors

We handled type checking as follows. First, we created more types than the usual ones in our programming language (integers, booleans, struct, etc.). These types serve two purposes: detecting errors and typing instructions. Instruction typing is used to locate instructions that are impossible in certain contexts. For example, a function declaration inside a struct, among others.

Each node receives a set of expected types. According to that set, it type-checks itself and its children. If the node type does not match any expected type, a type-checking error is raised showing the expected types. We repeat that these types may also be instruction types, such as DECVARIABLE (a variable declaration), among others. After being type-checked, each node returns upward a set of possible valid types for that node.

Let us now see some errors. We will not show all of them because there are many. At the end of this section we mention several text files prepared with the highlighted errors marked in comments.

Input [input14_errtipado.txt]

```
1  int () main() {
2    int (int ) f(a){};
3    bool (int ) f(a){};
4    f(2); //Error: ambiguous expression
5    void () f () {
6      return True; //Error: void functions do not return values
7    };
8    int  x;
9    //In structs, only variable declarations (initialized or not) and definitions
     of other new types are allowed
10   struct A {
11     int  a := 2; //Correct
12     struct B {};
13     void () f2() ; //Error: function declaration is not allowed inside a
     struct
14     x := 2; //Error: assignment (if not a declaration) is not allowed inside
     a struct
15   };
16 };
```

Output[2]

- `[Row 4, Column 2] Ambiguous expression`
- `[Row 6, Column 3] Type-checking error. Expected one of the following types: DEFSTRUCT DEFCLASS INSTRUCTION ASSIGNMENT DECVARIABLE DECFUNCTION`
- `[Row 13, Column 3] Invalid instruction`
- `[Row 14, Column 3] Invalid instruction`

It is during type checking that identifier binding is finally resolved. Declarations that do not match any expected type are removed from the binding set. If more than one binding remains, we raise an ambiguous-expression error. If no bindings remain, we also raise a type-checking error. This is a simple example.

Input [input15_errtipado.txt]

```
1 //These two functions are different because their return type is different
2 bool (int ) f(a){};
3 int (int ) f(a){};
4 int () main() {
5   f(2);//Ambiguity error, since any instruction type is expected
6   int  x := f(2); //Here it is resolved and there is no ambiguity error, since
    only the second function matches (because its return type is the one
    expected in the assignment)
7   int [2] arr := f(2); //Type-checking error because no declaration of
    identifier "f" matches the expected types
8 };
```

Output

- `[Row 5, Column 2] Ambiguous expression`
- `[Row 7, Column 16] Type-checking error. Expected one of the following types: INTEGER[2](INTEGER)`

The first error shows that, when a functional expression is treated as an instruction, the return type does not matter, and the compiler cannot distinguish between the two declarations of "f", so an ambiguity error occurs.

The second error shows that a functional type was expected with a two-dimensional integer-array return type and one integer parameter. This is because the return type was the expected type determined by the assignment and the parameter type by the parameters of that functional expression. However, there is no declaration of "f" with such a type, so an error is reported.

As already mentioned, to continue testing other type-checking errors, any of inputs 1, 2, and 3 can be used. These are longer code fragments with multiple type-checking errors. As stated, all errors are marked on the error line with a comment.

[2]Only type-checking errors are included

Chapter 5

Code Generation

Code generation is performed through the main method of constructorast by passing the file name as a parameter and uploading the generated .wat file to the generator folder. Then we convert the .wat file to wasm and execute it with the provided JavaScript function.

Code generation examples are split in two parts. On one hand, the source code is in the test folder, starting from input20.txt because the previous ones are error-detection tests. On the other hand, the generated .wat code goes to the generator folder, where the main.js file provided in the exercises is also located.

5.1 Code Examples

5.1.1 Example 1

Input20.txt in the tests folder is a simple example that can be compiled and then tested in the generator folder.

```
int x ;
int y ;
void () main() {
  x := 0 ;
  y := y + 1;
  prints(y);
};
int z ;
int w ;
```

5.1.2 Example 2

Input21.txt in the tests folder is a simple example similar to the previous one, but it adds a function call.

```
int x ;
int y ;
void () main() {
  x := 0 ;
  y := y + 1;
  x := f(y);
  prints(y);
  prints(x);
};
int z ;
int w ;
int(int) f(y) {
  z := y ;
  prints(z) ;
  y := y + 1 ;
  prints(y) ;
  return 2*z ;
};
```

5.1.3 Example 3

Input22.txt in the tests folder is a simple example similar to the previous one, but it adds a local variable.

void() main() x := 0; y := x + 1; int r; scans(r); prints(r); x := f(r); prints(y); prints(r); prints(y); prints(x); ; int z; int w; int(int) f(y) //shadows the global y z := y; prints(z); y := y + 3; prints(y); return 2*z; ;

```
int x ;
int y ;
void () main() {
  x := 0 ;
  y := y + 1;
  int r;
  x := f(y);
  prints(r);
  prints(y);
  prints(x);
};
int z ;
int w ;
int(int) f(y) {
  z := y ;
  prints(z) ;
  y := y + 3 ;
  prints(y) ;
  return 2*z ;
};
```

5.1.4 Example 4

Input23.txt in the tests folder is a simple example similar to the previous one, but the global variable is passed by value.

```
1  int x ;
2  int y ;
3  void () main() {
4    x := 0 ;
5    y := y + 1;
6    int r;
7    x := f(y);
8    prints(r);
9    prints(y);
10   prints(x);
11 };
12 int z ;
13 int w ;
14 int(int) f(y) {
15   z := y ;
16   prints(z) ;
17   y := y + 3 ;
18   prints(y) ;
19   return 2*z ;
20 };
```

5.1.5 Example 5

Input24.txt in the tests folder is a simple example similar to the previous one, but it uses arrays.

```
int x ;
int y ;
void () main() {
  x := 0 ;
  y := y + 1;
  int n;
  scans(n);
  prints(n);
  int[3] (a);
  a[0] := x;
  a[1] := y;
  a[2] := y + 1;
  x := f(a[n]);
  prints(a[2]);
  prints(a[n]);
};
int z ;
int w ;
int(int) f(y) {
  z := y ;
  prints(z) ;
  y := y + 3 ;
  prints(y) ;
  return 2*z ;
};
```

5.1.6 Example 6

Input25.txt in the tests folder is a simple example similar to the previous one, but it uses structs.

```
int x ;
int y ;
struct A {  int a  int b;
} ;
void () main() {
  x := 0 ;
  y := y + 1;
  int n;
  scans(n);
  prints(n);
  struct A s;
  s.b := n;
  s.a := 2;
  prints(s.a);
  x := f(s.a);
  prints(s.b);
  prints(s.a);
};
int z ;
int w ;
int(int) f(y) {
  z := y ;
  prints(z) ;
  y := y + 3 ;
  prints(y) ;
  return 2*z ;
};
```

5.1.7 Example 7

Input26.txt in the tests folder is an example that combines the two previous ones.

```
int x ;
int y ;
struct A {   int[3] a   int b;
} ;
void () main() {
  x := 0 ;
  y := y + 1;
  int n;
  scans(n);
  prints(n);
  struct A s;
  s.b := n;
  s.a[0] := x;
  s.a[1] := y;
  s.a[2] := y + 1;
  x := f(a[n]);
  prints(s.a[n]);
  prints(s.b);
  prints(s.a[n]);
};
int z ;
int w ;
int(int) f(y) {
  z := y ;
  prints(z) ;
  y := y + 3 ;
  prints(y) ;
  return 2*z ;
};
```

5.1.8 Example 8

Input27.txt in the tests folder is an example showing the use of switch and booleans.

```
1  void () main() {
2    int x := 1 ;
3    bool par := False ;
4    switch(x) {
5      case 1 {
6        par := False ;
7    }
8      case 2 {
9        par := True     }
10   }
11   if(par) {
12     prints(0);
13   } else(par) {
14     prints(1);
15   }
16 };
```

5.1.9 Example 9

Input28.txt in the tests folder is an example showing how methods work and modify class state.

```
class A {
  int x ;
  int() getX() {
    return x;
  };
  void(int) setX(a) ;
    x := a;
  };
  void() incrementX() {
    x := x + 1;
  };
void () main() {
  Class A s;
  s.x := 1 ;
  prints(s.x);
  prints(s.getX());
  s.incrementX() ;
  prints(s.x);
};
```

5.1.10 Example 10

Input29.txt in the tests folder is an example showing how pointers work.

```
1 void () main() {
2   int* p := null;
3   prints(p);
4   p := new;
5   prints(p);
6   prints(*p);
7   *p := 5;
8   prints(*p);
9 };
```

5.1.11 Example 11

Input30.txt in the tests folder is an example showing how loops work.

```
1 void () main() {
2   int n := 2;
3   int m := 3;
4   while(n > 0) {      n := n - 1;
5     c := c + m;
6   }   prints(c);
7 };
```

5.1.12 Example 12

Input32.txt in the tests folder is an example showing how recursive function calls work. It is important to note that parameter passing is by value, since otherwise it does not work because recursion gives a non-assignable element (a-1).

```
void () main() {
  int n := 3;
  if(esPar(n)) {      prints(0);
  } else {      prints(1);
  } };
bool($int) esPar(a) {    if(a == 0) {      return  True ;
  } elsif (a == 1) {      return False;
  } else {      return not esPar(a-1) ;
  } }
```

5.1.13 Final Notes

There is an example 31 about assigning arrays or structs and possible functions returning array, class, or struct types. In both cases we implemented the same approach: we assume the data structure is provided at the end of our static memory in the unassigned memory area and we use our copy-memory function. We only need to be able to copy from the stack pointer onward, which is done recursively in the codeB function. We only had debugging left to do, but we ran out of time. In addition, there are linking issues with inner functions (outside the global scope).